Dynamic In-node Group-aware Scheduling for Multi-tenant Machine Learning Services on Kubernetes

Peini Liu*†, Jordi Guitart*†
*Barcelona Supercomputing Center, Barcelona, Spain
†Universitat Politècnica de Catalunya, Barcelona, Spain
E-mail: {peini.liu, jordi.guitart}@bsc.es

2025 IEEE 18th International Conference on Cloud Computing (CLOUD) | 979-8-3315-5557-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/CLOUD67622.2025.00017

***Abstract*—Machine Learning (ML) is becoming pervasive and integrated into different kinds of businesses. Hosting multi-tenant ML services on a unified platform requires efficient orchestration. Service orchestration considers two levels: an in-cluster scheduling to decide the node allocation, followed by an in-node scheduling to manage the resource distribution within the node. Previous works introduced multi-container deployments for ML services, demonstrating that partitioning the ML service and enabling CPU/Memory affinity for each container improves performance. These multi-container deployments have been enabled for Kubernetes at in-cluster level to allocate multiple groups of containers across nodes. However, when the containers are launched in the node, the in-node scheduler lacks awareness of the group information which challenges those containers for a fine-grained resource allocation, especially when multiple groups of containers share the same node. This paper presents an in-node group-aware scheduling mechanism for multiple ML services, where each service scheduling contains a group resource selection and container resource assignment. Moreover, we also provide a dynamic resource controller (DRC) to dynamically reallocate the resource allocation for containers using the mechanism, which monitors the group changes and acts with the real containers' system cgroups adaptation. Our results show that for deploying ML services with the same ML model and different ML models, DRC throughput outperforms other deployment scenarios by up to 258% and 319% respectively. In an experiment deploying a dynamic workload of multiple mixed ML services, *DRC* outperforms baseline *NONE-Single* by 242%, 75%, and 28% for the average throughput of services with Mobilenet, Resnet50, and VGG16, respectively, and also DRC results in a makespan 44% faster than *NONE-Single* and 4% faster than *CM-Multi*.**

***Index Terms*—Group-aware scheduling, In-node scheduling, Affinity, ML Services, Cgroup, Kubernetes.**

I. Introduction

Nowadays, data and artificial intelligence (AI) have been utilized to empower all different kinds of areas, and more and more ML services are enabled for multiple businesses such as autonomous vehicles and transportation, e-commercial, healthcare, etc. Containerization has emerged as a fundamental technology for those services, because it packages applications with all their dependencies, ensuring portability across environments. It also improves resource utilization, flexibility, scalability, and enables agile developments and deployments within work teams.

Hosting multiple ML services on a unified cognitive computing continuum requires container orchestrators. The orchestrator has the capability to launch and manage containers and their entire lifecycle, while leveraging resource availability and user requirement specifications to decide the specific placement of containers. Kubernetes [1] is a dominant container orchestrator that has been extensively adopted in commercial production systems, such as Google Kubernetes Engine [2]. In terms of container scheduling, Kubernetes has a central scheduler within the control-plane to decide the node deployment for each Pod (i.e., in-cluster scheduling). Once a container is assigned to a node, there is a Kubelet agent at each node to determine how to launch the containers of the Pod in this node (i.e., in-node scheduling).

A large amount of work has been done with Kubernetes scheduler at the in-cluster scheduling level [3]–[5] for allocating containers to a node, but they did not consider fine-grained scheduling at the in-node level. In-node container scheduling must fine-tune how multiple containers share CPU, memory, cache, etc., after the orchestrator picks up a node. Currently, most systems, particularly the default Kubelet, rely on Linux cgroups and the kernel's Completely Fair Scheduler (CFS) to do this in-node scheduling. Moreover, there exist an advanced Kubelet agent that includes additional internal resource managers, such as CPU manager, topology manager, etc., to allocate and isolate in-node resources to each container. These resource managers ensure that each container operates within its resource limits by enforcing constraints through its assigned cgroup. However, the current in-node container resource allocation has some limitations. First, it lacks application awareness to consider the relationship among containers. It also lacks the capability to identify applications from multiple tenants.

Introducing a dynamic group-aware strategy at the in-node scheduling level can help to improve system resource utilization and application performance, especially in scenarios involving multi-container deployments of ML services, by optimizing the group coordination and resource affinity to enable more efficient service scheduling and execution. Several works [6]–[8] have demonstrated that partitioning the processes/threads belonging to an MPI/AI application into multiple containers can achieve better performance for computation-

2159-6190/25/$31.00 ©2025 IEEE
DOI 10.1109/CLOUD67622.2025.00017

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

intensive applications, and have experimented with corresponding CPU/memory affinity with multi-container deployment schemes. [4] has enabled group-aware scheduling for the Kubernetes scheduler at in-cluster level to allocate multiple containerized HPC applications among nodes. However, it remains challenging at in-node scheduling level to fine-tune the resource allocation for a group of containers belonging to a ML service, especially considering fine-grained CPU/memory affinity for each container. Moreover, when there are multiple ML services sharing the node, balancing among groups of containers belonging to each service is also challenging.

To this end, this paper presents a dynamic in-node group-aware scheduling, which can adapt the containers' cgroups considering the application container group awareness at runtime. Our main contributions are as follows:

- We enable multi-container deployment schemes for multi-tenant ML services on Kubernetes.
- We propose an in-node group-aware scheduling mechanism, which has the capability to provide balanced node resources among groups, and then enable resource affinity for containers within a group.
- We propose a dynamic resource controller (DRC) for controlling containers' cgroups, where a node agent is actively reacting to multiple group changes and updating containers' cgroups within this group accordingly.
- We evaluate DRC in a real Kubernetes environment for deploying homogeneous and heterogeneous multiple ML services. Also, we conduct DRC to deploy dynamic workloads comprising mixed ML services.

II. Background and Related Work

A. Container and Container Orchestration

Containerization is a lightweight virtualization technology that builds upon resource isolation and limitation features of the Linux kernel, such as `namespaces` and `cgroups`, respectively. Currently, containers are widely used to pack applications because of the benefits of portability, isolation, and high availability.

Numerous platforms support container orchestration, with Kubernetes [1] being dominant in the computing system. Kubernetes supports the orchestration of containers and provides resource management for its minimal unit Pod. From the users' perspective, they can submit simple specifications of applications that are encapsulated into containers and wrapped as Pods to be deployed in the nodes. From the providers' perspective, all the nodes and resources are controlled by Kubernetes. Upon receiving a deployment request, default Kubernetes scheduler [9] checks the Pod resource requirements, and selects the node (using a scheduling policy to filter and rank nodes) to run each Pod. Moreover, there are many custom schedulers in Kubernetes [3], for instance network-aware scheduling [5], telemetry-aware scheduling [10], disk-aware scheduling [11], and load-aware scheduling [12].

Some custom schedulers (*e.g.,* Volcano, Ray) enable Gang scheduling in Kubernetes, ensuring that a group of related Pods only start when all required resources are available [13] [14]. Similarly, [4] has enabled group-aware scheduling for Kubernetes scheduler at in-cluster level to allocate multiple containerized HPC applications among nodes. However, the group information is not considered within the node where the Pods are allocated.

In conclusion, these policies applied for container orchestration are at in-cluster level, which aim to select the node for each Pod, thus the in-node resource allocation is complementary to these policies in order to fine-tune the resource sharing of containers.

B. In-node Resource Allocation Policies

In Kubernetes, once the Pod is tagged with a node, the corresponding Kubelet agent in that node will take this information to launch the Pod on the selected node. By default, Kubelet does not enable specific Pod resource isolation, thus all Pods can use any resource in the node but within the specified resource limits and relying on CFS scheduling. In addition, native Kubelet supports some features like the CPU Manager and Topology Manager for more fine-grained resource allocation: (1) *CPU Manager*: when configured to "static", Kubelet will enable CPU pinning for containers that request whole CPU cores [15]; (2) *Topology Manager*: provides Non-Uniform Memory Access (NUMA)-aware resource allocation, ensuring that CPU and memory resources are allocated on the same NUMA socket for improved performance. It can be set as "best-effort" or "single-NUMA-node" [16]. Additionally, Kubernetes v1.27 released an alpha feature to enable in-place resource resizing for Kubernetes Pods [17].

Moreover, some custom policy enhancements have been developed in Cloud providers, such as HUAWEI Cloud, which provides an enhanced CPU policy that supports burstable pods allowing them to use certain CPUs to ensure application stability preferentially [18]. Also, Google Cloud highlights the performance benefits of implementing NUMA-aware scheduling to optimize resource allocation and reduce memory access latency, and the tradeoffs between optimizing for NUMA performance and reducing cache contention [19].

However, each of those Pods now is considered as an independent unit for in-node resource provisioning, thus none of the in-node scheduling policies considers the relationship of multiple Pods belonging to an application and establishes a group for them to enable holistic scheduling.

C. In-node Multi-container Deployments

Considering the Pods dependencies of an application can benefit several scenarios, a typical one is a multi-container deployment for HPC/AI applications which partitions the application processes or threads into multiple containers [20]. Some performance analyses [6] [7] indicate that an MPI application with multi-container deployment schemes can achieve better performance, and that setting CPU/Memory affinity for the containers help. Their latest work [8] presented the performance benefit of utilizing multi-container deployment schemes for an AI application and also the advantages of

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

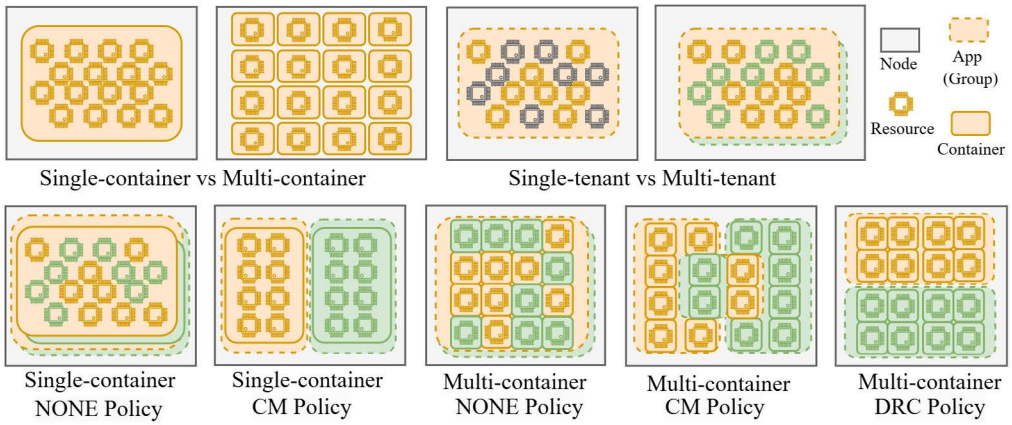

Fig. 1. Different deployment scenarios.

enabling CPU/Memory affinity for those containers on a Kubernetes cluster. However, this work uses Kubelet default policy and customized static policy to assign the containers' cpuset.cpus to a range of CPUs within a node before benchmarking, which does not have the ability to monitor a group of containers and adjust the CPU range allocation at runtime. Moreover, it did not evaluate multi-tenant ML services.

Some other works [21] [22] analyzed far-memory, where a custom scheduler could be aware of the memory bottleneck and change the cgroup of the memory node without changing the total amount of memory available. The improved swap system Fastswap and the cluster scheduler provide support for cluster-wide far memory, however the scheduler is customized and does not support containers.

Even though these works do not directly show how to update the cgroup of a container, they highlight some scenarios where updating cgroups (*e.g.,* especially for a range of CPU/memory cpuset.cpus, cpuset.mems) will potentially provide good performance for applications.

In conclusion, current in-node resource allocation requires a scheduling policy that considers the application-awareness of multiple containers established as a group and the support for fine-tuning deployments of multi-tenant ML services for holistic scheduling. In addition, it requires a dynamic resource controller that can actively monitor a group event and react with a cgroups update.

III. Basics of Deployment Scenarios

The basic deployment models for ML services are introduced in Figure 1. In this paper, we contrast single-container with multi-container deployments for an ML service, as well as single- vs. multi-tenant scenarios. In this section, first we define these scenarios (shown in the top part of Figure 1) and then we present different deployment schemes (shown in the bottom part of Figure 1).

- **Single-container vs. Multi-container**: A single-container deployment is the basic deployment for an ML service, where the scheduler allocates resources to the service runtime and the runtime schedules the workers to use them. A multi-container deployment for an ML service is considered for more fine-grained resource allocation, where the ML service can distribute its load into multiple backends, and the scheduler can allocate resources to each of the containers. Detailed thread partition and resource allocation for multiple containers can be found in [8].
- **Single-tenant vs. Multi-tenant**: A single-tenant scenario refers to an ML service on a platform without resource sharing with other ML services from other tenants. Whereas, a multi-tenant scenario means that ML services from different tenants are sharing resources from the same platform.

In this paper, we consider a multi-tenant in-node scheduling, hence our platform will be a single node that is shared by multiple ML services from different tenants, and the objective is to efficiently deploy multiple groups of containers. The bottom part of Figure 1 shows examples of different deployment schemes for multiple groups of containers, which are detailed below. As an example, we deploy two ML services in a node with 16 cores and 2 NUMA sockets.

- **None-Single**: Each group (i.e., service) has only one container that specifies a CPU request of 8. Each container can use 8 CPUs from all the 16 available cores. The CFS scheduler makes the in-node scheduling.
- **CM-Single**: Each group has only one container that specifies a CPU request of 8. Each container is assigned the requested amount of resources with CPU/Memory affinity (*e.g.,* 8 dedicated cores, memory from a single NUMA socket). The Kubelet policy makes the resource pinning for each container.
- **None-Multi**: Each group has multiple containers, in this example 8 containers, each one requesting 1 CPU. Similar as NONE-Single, each container can request 1 CPU from all 16 CPUs. The CFS scheduler does the in-node scheduling.
- **CM-Multi**: Each group has multiple containers (*i.e.,* 8 containers, each one requesting 1 CPU). Each container is assigned resources with CPU and memory affinity

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

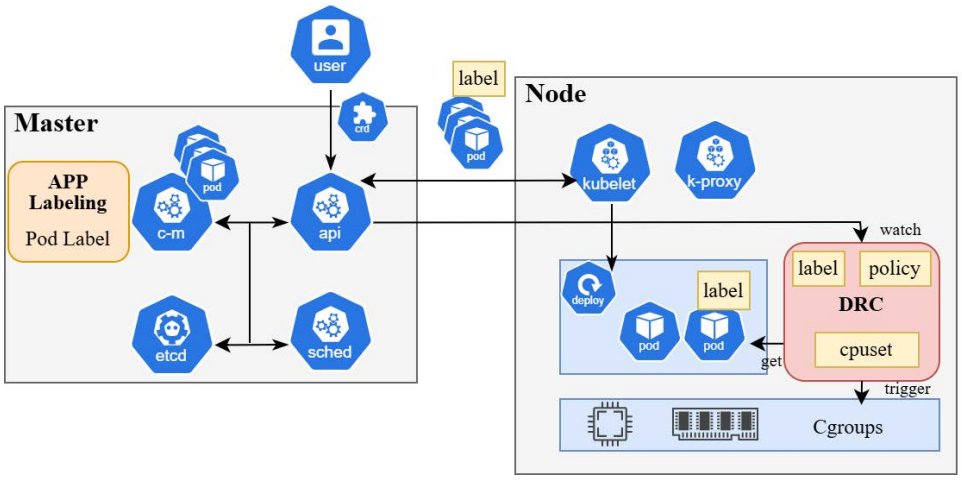

Fig. 2. System architecture for resource allocation on Kubernetes.

(*e.g.,* 1 dedicated core and memory from a single NUMA socket). The Kubelet policy assigns resources to each container based on their startup sequence and allocates CPU cores in numerical order without group awareness.

However, enabling group-aware scheduling for multi-container deployments in a multi-tenant scenario is important because of two considerations. First, multi-tenant groups can be heterogeneous (i.e., different ML models), and this diversity creates different resource contention on the shared node, especially in cache and memory bandwidth. If the system allocates resources unevenly to the group, it may lead to inconsistent performance degradation in different NUMA sockets. Second, multiple containers in the same group are homogeneous with a synchronized life-cycle, generating system loads together. Resource balancing is also important because the system load will significantly change in the node once a group is added or deleted, especially in a dynamic scenario.

To this end, our **DRC Policy** aims to enable group awareness for the dynamic multi-tenant multi-container scenario. For each scheduling step, it first selects resources for a group ensuring load balancing among NUMA sockets, and then allocates resources to containers within that group using resource affinity for efficient resource usage. DRC module and policy are explained in detail in Section IV.

IV. Dynamic Resource Controller

In this section, we first introduce a system architecture for resource allocation on Kubernetes. After that, we present the Dynamic Resource Controller (DRC) module, and define its sensors and actuators, and finally we propose a group-aware scheduling strategy and enable dynamic cgroup control.

A. System Architecture

Figure 2 presents the system architecture for resource allocation on Kubernetes. First, it shows how Kubernetes deploys Pods, that is, a user specifies an application, then the controller manager wraps the application as a single or multiple Pods and the scheduler decides the node placement of these Pods. Once those Pods are assigned to a node, the Kubelet agent in each node will utilize its CPU/Topology Manager to decide resource usage and start the Pod by calling the container runtime to create cgroups for allocating specific resources.

DRC works after the first Pod initialization thus has no conflict with Kubelet. As shown in the right of Figure 2, the DRC module operates as a node agent which can provide dynamic in-node group-aware scheduling. From an abstract point of view, its primary role is to continuously watch changes at both the application and the infrastructure layers, automatically triggering group-aware scheduling policy to adapt to those changes by holistically allocating resources for applications. In particular, after an application is allocated to a node by the Kubernetes scheduler, the in-node DRC will balance resources among multiple application groups, and will also enable resource allocation adjustment by changing cgroup settings of containers within an application group. This mechanism is especially critical for a multi-container deployment scenario and in multi-tenant environments, where resource sharing is common. The dynamic and fine-grained management of resource allocations among application groups and within an application group ensures that each container receives the appropriate resources based on the latest application and infrastructure state.

B. Dynamic Resource Controller Module

DRC Module is an agent that dynamically adapts to the group changes in a node. The detailed sensors and actuators of the DRC module are explained below.

Table I shows the sensors of DRC from two layers (*i.e.,* Application and Infrastructure). In the application layer, the agent is capable to get two levels of group changes. One is on the number of ML services, when a new group is generated or a group is deleted, the policy is triggered to select or release resources. Another one is on the number of containers within a group, if the total amount of containers reach the desired numbers within the group, the policy is triggered

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DRC SENSORS

	Layer	Metrics
Sensor	Application	Number of groups: Multi-tenant Number of containers: Multi-container
	Infrastructure	Available CPUs and NUMA socket resources

TABLE II
DRC ACTUATORS

	Control Point	Parameters
Actuator	cpuset	cpuset.cpus cpuset.mems

to select resources for the group and allocate resources to containers within the group. In the infrastructure layer, the current available CPUs/memory resources and the resource topology information is able to be monitored.

Table II shows the actuators of DRC, which is responsible for adjusting the cgroups control of all containers within the node. In practice, editing the cgroup is not recommended because it may interfere with Kubernetes' resource management. However, we noticed that native Kubernetes only decides on the total amount of the CPU/memory resources allocated to a container. Our proposed actuators will not use DRC to control the total amount of resources but the specific allocated resources, hence there will be no conflict. Given the benefit we see in the literature to use the proper CPU/memory affinity, we propose to have control points on the CPUSET, where the containers can use a range of CPUs and specific memory nodes.

C. Dynamic Group-aware Scheduling Mechanism

Finally, we propose a group-aware scheduling mechanism for the deployment of ML services using the sensors and actuators mentioned in Section IV-B. The notations used for explaining the problem and algorithms are shown in Table III.

In theory, the proposed group-aware scheduling mechanisms can be enabled by any container orchestrator at in-node level by changing the implementation of the sensor to get information of groups and the actuator to adapt the containers' cpuset. However, in this work, we focus on the Kubernetes platform, given its dominance in container orchestration.

We assume a Cloud platform (*e.g.,* Kubernetes) shared by different tenants to provide their ML model as a ML service. Hence, we have all the ML services (from different tenants) $S = \bigcup_{i=1}^{n} S_i$. For each ML service S_i, we enable both single- or multiple- container deployment, allowing the service to be deployed as a container group G_i. The container group is defined as $G_i = \bigcup_{j=1}^{m} p_{i,j}$, consisting of multiple Pods in the Kubernetes concept. The objective is to find for G and P the resource allocations M_G and M_P.

The initial scheduling for ML services $\mathcal{S}$ is handled by Kubernetes, where the default scheduler selects nodes for multiple groups of Pods. The Kubelet then launches the Pods on the assigned node, so that the node has a set of init cgroups $\mathcal{C}$ to control the resource allocation for each Pod.

TABLE III
NOTATION TABLE.

Notation	Explanation
$\mathcal{S}$	A set of ML services.
$\mathcal{G}$	A set of PodGroups.
$\mathcal{P}$	A set of Pods.
S_i	Service i.
G_i	PodGroup i.
$p_{i,j}$	Pod j in Service i.
$\mathcal{A}$	Pod CPU request.
$\mathcal{E}$	Replicaset Event
$\mathcal{N}$	Topology information.
$\mathcal{U}$	Available CPUs and Memory resources with topology information.
$\mathcal{L}$	System Pod cgroup path.
$\mathcal{C}$	A set of Cgroups, each cgroup has its cpuset-cpus and cpuset-mems.
$\mathcal{C}_\mathcal{G}$	A set of Cgroups for PodGroups. $\mathcal{C}_\mathcal{G} = \{C_1, C_2, \ldots, C_i\}$
$\mathcal{C}_\mathcal{P}$	A set of Cgroups for Pods. $\mathcal{C}_\mathcal{P} = \{C_1, C_2, \ldots, C_i\}$
M_G : $mg \mapsto$ (cpuset-cpus, cpuset-mems)	Mapping of podgroup i allocated to a cgroup of cpuset-cpus, and cpuset-mems.
M_P : $mp \mapsto$ (cpuset-cpus, cpuset-mems)	Mapping of pod j in group i allocated to a cgroup of cpuset-cpus, and cpuset-mems.

Algorithms 1 to 3 specify the overall group-aware scheduling algorithm. The DRC uses them to dynamically reschedule the cgroups (especially cpuset.cpus, cpuset.mems) for each group G_i and Pods within the group $p_{i,j}$.

Algorithm 1 Group-aware Scheduling Algorithm

Input: Running pods $\mathcal{P}$. Topology info (NUMA sockets, available CPU list) $\mathcal{N}$. Real-time event of Replicaset $\mathcal{E}$.
Output: A group of $\mathcal{P}$ with updated set of cgroups $\mathcal{C}$ of CPU and memory allocations.

Initialize: $\mathcal{U} \leftarrow$ SetCPUsList($\mathcal{N}$)
Initialize: $\mathcal{S}$
while replicaset event $e \in \mathcal{E}$ **do**
  $S_i \leftarrow Replicaset$
  if $e.type$ == "added" or "modified" **then**
    if S_i.replicas == S_i.available_replicas **then**
      $G_i \leftarrow$ grouppods_w_label("`dyn-cpuset`",S_i)
      $M_{G_i}[M_{P_{i,j}}], \mathcal{U} \leftarrow$ process_cgroups_allocation(G_i,$\mathcal{U}$). ▷ **Algorithm 2: Cgroups Reallocation for PodGroup Policy**
      add_service($\mathcal{S}$,S_i,$M_{G_i}[M_{P_{i,j}}]$)
    end if
  else if $e.type$ == "deleted" **then**
    $\mathcal{U} \leftarrow$ release_cgroups_allocation(S_i,$M_{G_i}[M_{P_{i,j}}]$))
    delete_service($\mathcal{S}$,S_i)
  end if
end for

Algorithm 1 is the main scheduling algorithm within the DRC. First, the DRC has information of the node topology

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

$\mathcal{N}$, available CPUs/memory resources with NUMA socket topology information $\mathcal{U}$, and initialized service $\mathcal{S}$. Then, the DRC continuously watches the group changes: when there is a new service added, Kubernetes will process an initial deployment; when the service is deleted the resources will be released. More importantly, when each group is ready with its number of Pods, DRC aggregates all the Pods for the group and calls for a cgroups reallocation for the group and the pods, as shown in Algorithm 2.

Algorithm 2 Cgroups Reallocation for PodGroup Policy

Input: Pod Groups G_i; Available CPUs $\mathcal{U}$ with NUMA topology.

Output: Updated Cgroup allocations: a list of mapping $M_{G_i}[M_P]$.

1: Get total CPU request of the Group: $A_j = \sum_{p \in G_i} r_p$

2: Evenly distribute A_j across the NUMA sockets:

1) Let $\mathcal{N} = \{N_1, N_2, \ldots, N_k\}$ be the NUMA sockets.
2) Assign $\lfloor A_j/k \rfloor$ or $\lceil A_j/k \rceil$ CPUs to each node, respecting node capacity.
3) Select specific CPU cores from each node to minimize fragmentation.
4) Store assigned CPUs for group G_i in $M_{G_i}[mg_i]$

3: **for** each pod $p_{i,j} \in G_i$ **do**

4: Compute $p_{i,j}$'s share of G_i's Cgroup allocation:

$$\text{cpuset.cpus}_{p_{i,j}} = \frac{M_{G_i}[mg_i] \mapsto \text{cpuset.cpus}}{len(G_i)}$$

$$\text{cpuset.mems}_{p_{i,j}} = \frac{M_{G_i}[mg_i] \mapsto \text{cpuset.mems}}{len(G_i)}$$

5: Update $M_{P_{i,j}}$ with assigned cpuset.cpus and cpuset.mems:

$$M_{P_{i,j}}[mp_{i,j}] \leftarrow \text{cpuset.cpus}_{p_{i,j}}, \text{cpuset.mems}_{p_{i,j}}$$

6: **end for**

7: AllocateCgroups(G_i,$M_G[M_P]$). ▷ **Algorithm 3: Container Cgroups Assignment**

$\mathcal{U} \leftarrow \mathcal{U}$ - $M_{G_i}[M_{P_{i,j}}]$

8: **return** $M_{G_i}[M_{P_{i,j}}], \mathcal{U}$

Algorithm 2 shows the cgroup reallocation for a group of pods. It contains two stages for the allocation, first, select resources for the group, and then assign resources for each pod within the group. In the group resource selection stage, we begin by aggregating the resource requests of all Pods within the group to determine the total demand (line 1). Next, resources are evenly allocated from all NUMA sockets to the group (line 2), ensuring that the selected resources align with the available resources and system NUMA topology. Once the group's resources are reserved, the policy further enables CPU and memory affinity for each Pod (line 3-5), so each Pod is allocated with new specific cgroups (*i.e.,* cpuset.cpus, cpuset.mems).

Algorithm 3 shows the actuator to do the final cgroups assignment. First, the DRC should get the pod and get and filter its cgroups path (line 3). This implementation can be different within different platforms. For Kubernetes the Pod cgroups will be in `/sys/fs/cgroup/kubepods.slice/`, and should be filtered with the ids. Finally, overwrite new cgroups allocation from Algorithm 2 to the corresponding cgroups. To note, this path should also be mounted to the DRC agent in order to enable the final action.

Algorithm 3 Container Cgroups Assignment

Input: Pod Groups G_i, Cgroups mapping for pod group $M_{G_i}[M_{P_{i,j}}] : mp \mapsto (\text{cpuset-cpus}, \text{cpuset-mems})$

Output: Updated container CPU and memory allocations

1: $mp_{i,j} \leftarrow M_{G_i}[M_{P_{i,j}}]$

2: **for** each pod $p_{i,j} \in G_i$ **do**

3: $\mathcal{L}_{p_{i,j}} \leftarrow$ GetSystemCgroupsPath($p_{i,j}$)

4: Allocate $p_{i,j}$'s Cgroups allocation:

$$\mathcal{L}_{\text{cpuset.cpus}_{p_{i,j}}} = M_{Pi,j} : mp_{i,j} \mapsto \text{cpuset.cpus}$$

$$\mathcal{L}_{\text{cpuset.mems}_{p_{i,j}}} = M_{Pi,j} : mp_{i,j} \mapsto \text{cpuset.mems}$$

5: **end for**

V. Evaluation

In this section, we evaluate our DRC system and the group-aware scheduling mechanism on top of the Kubernetes platform. The DRC agent is able to perform dynamic fine-grained cpu/memory allocation for image classification ML inference services on a node.

A. Experimental Settings

Hardware: Our experiments are executed on a machine with 2x Intel® Xeon® Platinum 8360Y CPU @ 2.40GHz, 256 GB RAM. Each CPU has 36 physical cores and with simultaneous multi-threading disabled and with the governor for the dynamic voltage and frequency scaling set to a fixed maximum frequency in mode 'performance'.

Software: The machine runs Ubuntu 22.04 (64 bits) with host kernel 6.5.0-44-generic. The Kubernetes platform is built through minikube with Kubernetes version v1.28.14, with docker v27.2.0 as driver. Models' directory is mounted to the minikube container. Minikube starts with two different settings for the in-node Kubelet. Detailed settings for scenarios NONE and CM are listed in Table IV. 65 cores are available for Kubernetes applications, others are reserved for system processes and Kubernetes core services. Our DRC can use up to 1 core, we can dedicate 64 cores for serving multi-tenant ML services.

TABLE IV
Minikube Kubelet Settings

Scenario	Settings
None	–extra-config=kubelet.system-reserved=cpu=4 –extra-config=kubelet.kube-reserved=cpu=3 –extra-config=kubelet.cpu-manager-policy=none
CM	–extra-config=kubelet.system-reserved=cpu=4 –extra-config=kubelet.kube-reserved=cpu=3 –extra-config=kubelet.reserved-cpus=0-3,68-70 –extra-config=kubelet.cpu-manager-policy=static –extra-config=kubelet.topology-manager-policy=best-effort

Model Serving Services: We choose computer vision applications with their associated reference models. Example vision applications include photo searches, object sorting,

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SUMMARY OF USED MODEL

Model	Single Stream Latency (CPU)	Parameters	Depth	Features
MobileNet	10-30ms	4.3 million	55	Optimized for low-latency, edge, and mobile applications
ResNet-50	50-200ms	25.6 million	107	Balances accuracy and latency, high computational cost
VGG-16	500ms-1s	138.4 million	16	High accuracy; high latency; computationally expensive; memory-intensive

and object detection. In particular, we use well-known vision models Resnet50, MobileNet, and VGG-16 from Keras applications [23]. Their basic information and features as shown in Table V. As the serving runtime we choose Tensorflow Serving. For single-container deployments we let the runtime to utilize the default setting, whereas for multi-container deployments we use the methods from this paper [8] to establish resource and threads splitting among containers. In this paper, each service is a tenant that is capable of serving multiple requests and demands 16 cores in all kinds of deployment scenarios.

Client: The client we use is ab [24] – Apache HTTP server benchmarking, which can particularly show how many requests per second our model service is capable of serving. We set the concurrency requests to 128, 96, and 64 for MobileNet, Resnet50, and VGG-16, respectively, because of their latency reference.

TABLE VI
SCENARIOS SETTINGS.

Scenarios	Kubelet	Granularity
NONE-Single	default	
NONE-Multi	default	multi-container deployments granularity
CM-Single	–cpu-manager-policy=static and –topology-manager=best-effort	
CM-Multi	–cpu-manager-policy=static and –topology-manager=best-effort	multi-container deployments granularity
DRC	DRC agent with label-aware cgroup control	multi-container deployments granularity

Evaluation Deployment Scenarios: We consider five scenarios. First of all, *NONE* is the baseline scenario, which uses the default settings of Kubernetes thus without containers' in-node resource isolation, meaning all containers will have as cpuset the entire node. As for *CM* scenario, we enable default Kubelet CPU management to explore CPU/memory affinity for each container. Detailed Kubelet settings are in Table IV.

By default, a ML service uses a single-container deployment, we record them as scenario *Single*. However, given the benefit of using multi-container deployments for online learning services [8], we also enabled *Multi* scenarios by splitting the services into multiple containers while reducing the corresponding number of threads. Note that, the total amount of requested resources are the same for both scenarios.

Scenario *DRC* maintains the benefits of the multi-container deployment granularity and CPU/memory affinity, and enables our proposed cgroup scheduling and assignment algorithms (see Algorithms 1-3) in the infrastructure Kubelet. Summarizing, the settings of the five scenarios are as shown in Table VI.

B. Experiment 1: Deployment of multi-tenant services serving the same type of ML model

Firstly, we set an experiment with a single type of application (i.e., MobileNet). In this experiment, we deploy 4 ML services each requesting 16 cores in all 5 deployment scenarios. Each service has a client continuously sending image processing requests at a concurrency level 128. The clients come with an arrival time interval of 5s and run for 30s. The entire experiment is repeated 10 times. The throughput and latency of these services are shown in Figure 3.

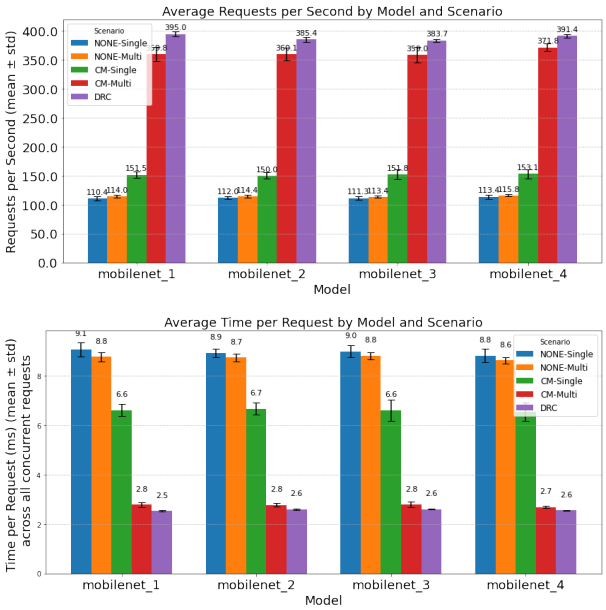

Fig. 3. Experiment 1: Throughput and latency of 4 MobileNet services.

As shown in the plots, *NONE-Single* and *NONE-Multi* have low requests per second (RPS) compared to other scenarios. This is because in the *NONE-** scenarios, a single container starts on all CPUs within the host, the Tensorflow Serving runtime creates as many threads as visible CPUs (i.e., 72) within this container but (on average) only 16 cores can be used by each container in *NONE-Single* and only 1 core can be used by each container in *NONE-Multi*, respectively. Thus, compared to *CM-** scenarios, where a single container starts on 16 cores and threads, or multiple containers start on 1 core

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

and thread, *NONE-** have more CPU migrations and context switches among more threads. This is a significant reason that *CM-** outperform *NONE-** up to 258%. Another factor which contributes to the performance improvement is the CPU and memory affinity. In scenarios *CM-**, containers are bound to specific CPUs and a memory node, thus have better cache utilization (less L3 misses), more local memory accesses, and less remote memory accesses than *NONE-** scenarios.

When enabling multi-container deployments granularity, we partition each service within a large number of containers, but fewer number of threads per container. *CM-Multi* is configured with a granularity of 16 containers per service, the total CPU requirement per container becomes equivalent to a single core. By assigning each container to a dedicated CPU (explicitly pining their threads to the core), we avoid threads migration and context switches, thus containers can exploit the CPU better. This is demonstrated from the results where *CM-Multi* has 137%-146% performance improvement regarding *CM-Single*.

We also see in the results that *DRC* throughput outperforms *CM-Multi* up to 9.8%. This improvement comes from the difference of how resources are allocated. In *CM-Multi*, by default the Kubelet enables the cpu-manager/topology-manager and treats each container as an isolated deployment unit and statically assigns a group of CPUs and memory to each container when it is generated. This static allocation ignores the relationships among containers belonging to the same services. In contrast, *DRC* leverages Pod label to identify groups of related containers. It first reserves the CPU and memory resources for the entire group and then distributes them among the containers that comprise it. Through this group-aware approach, DRC can keep the benefit of using CPU and memory affinity for containers within a group, and dynamically ensure that resources among groups are carefully balanced from the system.

C. Experiment 2: Deployment of multi-tenant services serving different types of ML models

Secondly, we set an experiment with different types of ML applications (i.e., 1 application of MobileNet and VGG16, and 2 applications of Resnet50). The settings are the same as experiment one, except for the client requests concurrency levels of the different services, namely 128 for MobileNet, 96 for Resnet50, and 64 for VGG16, respectively, due to the latency reference of the different models. The entire experiment is repeated 10 times. The throughput and latency of these services are shown in Figure 4.

For model mobilenet, resnet50_1, resnet50_2, and vgg16, *NONE-Multi* vs. *NONE-Single* has performance difference of 91%, -7%, -3% and -8%, respectively. Only mobilenet shows better performance in the *NONE-Multi* because it is a more lightweight model in comparison to the others. Even if multiple containers create more threads for using resources, those threads can spend more time doing productive computation and less time waiting on shared locks or cache lines. However, for a heavier model, each thread can place significant pressure on caches. Larger number of threads from the multi-container deployment will generate more cache misses causing a lower CPU utilization. In addition, *CM-** scenarios are better than *NONE-** up to 319% because in *CM-** scenarios the deployment scheme defines the appropriate number of threads and the CPU and memory affinity for each container instead of using the default settings as done in *NONE-**.

CM-Multi has a performance improvement of 241%, 17%, and 15%, and a performance degradation of -19% for models mobilenet, resnet50_1, resnet50_2, and vgg16, respectively, compared to *CM-Single*. CPU-intensive small models such as Mobilenet and Resnet50 show better RPS in *CM-Multi*, because multi-container deployments allow to exploit the CPU affinity better avoiding inefficient thread migrations and context switches. However in a multi-tenant context, *CM-Multi* will incur more resource usage in shared cache and memory than *CM-Single* for complex memory-intensive models such as VGG16. Those containers are more vulnerable to the performance impact of shared cache and memory contention.

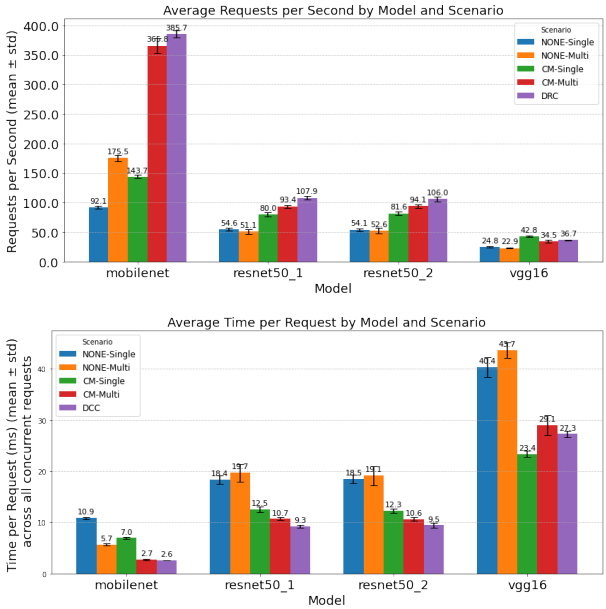

Fig. 4. Experiment 2: Throughput and latency for a mix of 4 services.

DRC shows a performance improvement of 5%, 16%, 13%, and 6% for models mobilenet, resnet50_1, resnet50_2, and vgg16 regarding *CM-Multi*. This is essentially because *DRC* better balanced the pod groups within the system. On the one hand, multi-tenant ML services can benefit from the CPU and memory affinity to enhance CPU usage as in *CM-Multi*. However, *DRC* can balance Pod groups among NUMA sockets, which can balance overall cache and memory contention among all multi-tenant collocated services, thus contributing to the overall performance.

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

D. Experiment 3: Deployment of a dynamic workload of multi-tenant services serving different types of ML models

Finally, we assess the effectiveness of our DRC agent and the group-aware resource allocation policy for scheduling a dynamic workload of ML services with different types of models. Instead of executing only 4 services co-located in a node within a given time period and statically evaluating the group-aware scheduling policy, this experiment explores a more dynamic scenario with changing groups. In this setting, multiple ML services arrive at different time and serve a given amount of requests.

In particular, we generated a total of 12 ML services distributed evenly across three ML models—ResNet50, MobileNet, and VGG16—resulting in four service instances per model type. ML services run with a random sequence, and we appended priorityClass to keep the order of services in all deployment scenarios. This setup can be further enhanced by employing a gang-scheduling approach, ensuring sufficient resources are allocated for all pods of a ML service before the scheduler applies their placement. Moreover, each service of one type of model will serve a given amount of requests. The client requests concurrency levels remain the same as used in Experiment 2: MobileNet, Resnet50, and VGG16 services with concurrent requests of 128, 96, and 64, respectively.

Figure 5 shows the average throughput of different types of services. *DRC* outperforms baseline *NONE-Single* by 242%, 75%, and 28% for Mobilenet, Resnet50, and VGG16, respectively. This is because most of the performance insights obtained in the previous experiments still hold for the *DRC* with a larger amount of services.

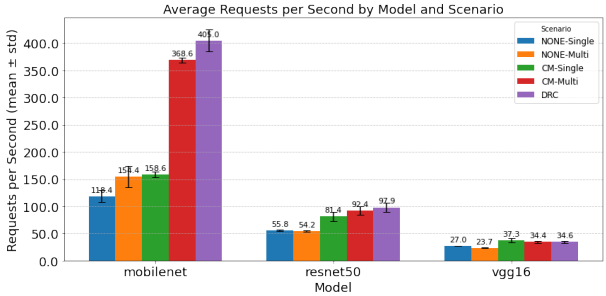

Fig. 5. Experiment 3: Average throughput of different types of services.

The details of the runs can be seen in Figure 6. First, this experiment shows that DRC works in a dynamic environment. The DRC is capable of sensing the group changes (*e.g.,* application added, deleted and composed by multiple containers) and reacts with the cgroup adaptation by using the group-aware scheduling policy. Also, the makespan shows that *DRC* is 44% faster than *None-Single* baseline scenario because of several benefits that we analyzed before (i.e., number of threads, CPU and memory affinity and multi-container deployment, as well as the pod group-aware load-balancing). These can be demonstrated from the results where *NONE-Single* is 19% faster than *CM-Single*, *CM-Multi* is 28% faster than *CM-Single*, *DRC* is 4% faster than *CM-Multi*.

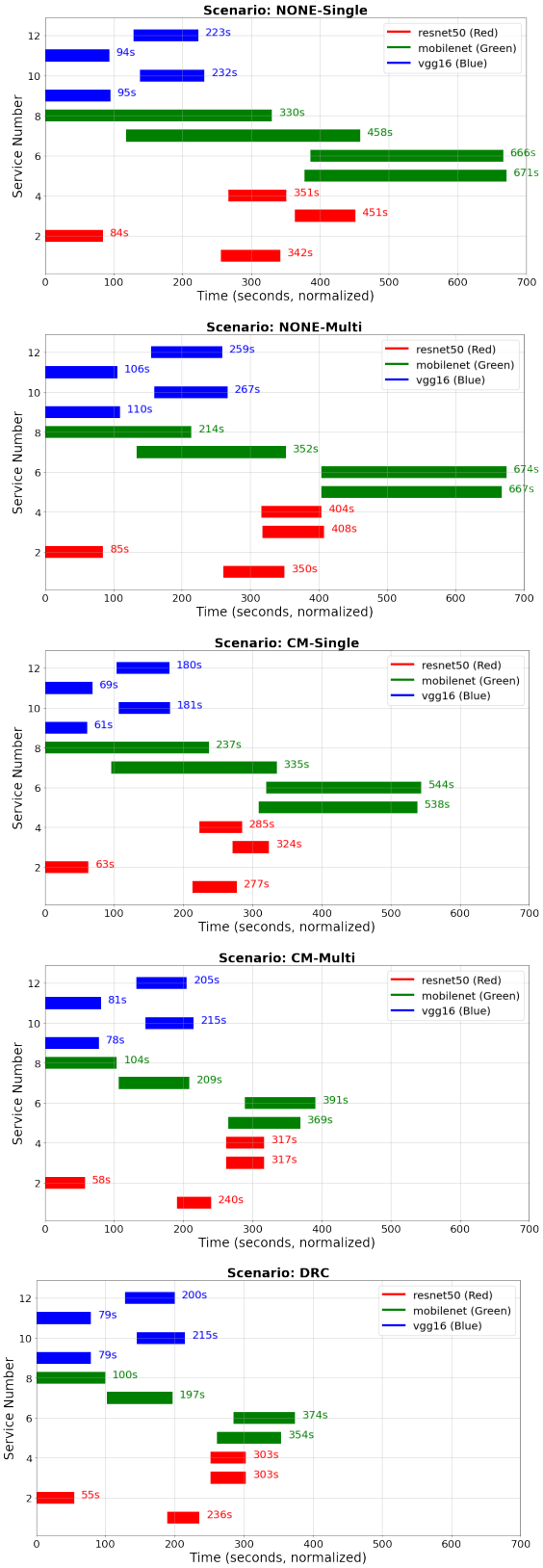

Fig. 6. Dynamic workload experiment: Makespan of serving multiple different services.

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion and Discussion

In this paper, we have first analyzed and introduced various deployment scenarios, then we have presented an in-node group-aware scheduling policy, which considers two stages of scheduling (*i.e.,* group resource selection and pod resource assignment), and finally we have proposed DRC - a Dynamic Resource Controller for Kubernetes - that dynamically manages Pods cgroups. DRC is composed by sensors to detect the group changes, the proposed policy to determine new cgroups and reallocate cgroups of containers accordingly. In the scenario with a dynamic workload of multiple ML services, the proposed scheduling with *DRC* is up to 44% faster than the basic *None-Single* baseline.

Several aspects are worth discussing. First, the presented scheduling policy can be adapted to other orchestration platforms by changing the implementation of group sensing in the service deployment, and actuator to enforce the cgroups changes. Second, even though currently our evaluation of DRC is within one node, DRC is a node agent and has the capability to scale to different nodes as well. Finally, as we mentioned this in-node scheduling is complementary to the in-cluster scheduling, thus it could be interesting to combine the two-level scheduling (*i.e.,* this work and also the work [4], which introduces the group-aware scheduling in a cluster) in a large-scale experiment.

Furthermore, we also infer that our strategy together with multi-container deployments can be advantageous for a wide range of computation-intensive workloads, not limited to image classification ML models but also others such as natural language processing (e.g., Transformer models). Future work may involve analyzing additional system resources—such as network and storage I/O—that can also impact the performance of ML services in some cases.

Acknowledgment

This work is financed by the EU-HORIZON programme under grant agreements EU-HORIZON GA.101092646, EU-HORIZON MSCA GA.101086248, by Generalitat de Catalunya (AGAUR) GA.2021-SGR-00478, and the Spanish Ministry of Science (MICINN), the Research State Agency (AEI) and European Regional Development Funds (ERDF/FEDER) PID2021-126248OB-I00, MCIN/AEI/10.13039/501100011033/FEDER, UE.

References

[1] Kubernetes, "Production-Grade Container Orchestration," 2022. [Online]. Available: https://kubernetes.io/

[2] Google, "Google Kubernetes Engine (GKE)," 2022. [Online]. Available: https://cloud.google.com/kubernetes-engine

[3] Z. Rejiba and J. Chamanara, "Custom scheduling in kubernetes: A survey on common problems and solution approaches," *ACM Comput. Surv.*, vol. 55, no. 7, Dec. 2022. [Online]. Available: https://doi.org/10.1145/3544788

[4] P. Liu and J. Guitart, "Fine-grained scheduling for containerized hpc workloads in kubernetes clusters," in *2022 IEEE 24th Int Conf on High Performance Computing Communications; 8th Int Conf on Data Science Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud Big Data Systems Application (HPCC/DSS/SmartCity/DependSys)*, 2022, pp. 275–284.

[5] J. Santos, C. Wang, T. Wauters, and F. De Turck, "Diktyo: Network-aware scheduling in container-based clouds," *IEEE Trans. on Netw. and Serv. Manag.*, vol. 20, no. 4, p. 4461–4477, Apr. 2023. [Online]. Available: https://doi.org/10.1109/TNSM.2023.3271415

[6] P. Liu and J. Guitart, "Performance comparison of multi-container deployment schemes for HPC workloads: an empirical study," *Journal of Supercomputing*, vol. 77, pp. 6273–6312, 2020.

[7] ——, "Performance characterization of containerization for HPC workloads on InfiniBand clusters: an empirical study," *Cluster Computing*, vol. 25, pp. 847–868, 2022.

[8] P. Liu, J. Guitart, and A. Taherkordi, "Performance characterization of multi-container deployment schemes for online learning inference," in *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, 2023, pp. 330–340.

[9] Kubernetes, "Kubernetes Scheduler," 2024. [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/

[10] Intel Corporation, "Telemetry-aware scheduling," https://github.com/intel/platform-awarescheduling, 2020, enabling Kubernetes to Make Intelligent Pod Placement Decisions with Platform Telemetry. [Online; accessed 2023-08].

[11] Kubernetes SIGs, "Disk i/o-aware scheduling (kep-624)," https://github.com/kubernetes-sigs/scheduler-plugins/tree/master/kep/624-disk-io-aware-scheduling, 2023, [Online; accessed 2023-08].

[12] ——, "Trimaran: Real load-aware scheduling (kep-61)," https://github.com/kubernetes-sigs/scheduler-plugins/tree/master/pkg/trimaran, 2021, [Online; accessed 2023-08].

[13] Ray, "Gang Scheduling with RayJob and Kueue," 2024. [Online]. Available: https://docs.ray.io/en/latest/cluster/kubernetes/examples/rayjob-kueue-gang-scheduling.html

[14] Volcano, "Plugins: Gang scheduling strategy," 2024. [Online]. Available: https://volcano.sh/en/docs/plugins/#gang

[15] Kubernetes, "Control CPU Management Policies on the Node," 2024. [Online]. Available: https://kubernetes.io/docs/tasks/administer-cluster/cpu-management-policies/

[16] ——, "Control Topology Management Policies on a node," 2024. [Online]. Available: https://kubernetes.io/docs/tasks/administer-cluster/topology-manager/

[17] ——, "Resize CPU and Memory Resources assigned to Containers," 2024. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/

[18] H. Cloud, "Enhanced CPU Policy," July 2024. [Online]. Available: https://support.huaweicloud.com/eu/usermanual-cce/cce_10_0552.html

[19] L. Tang, J. Mars, X. Zhang, R. Hagmann, R. Hundt, and E. Tune, "Optimizing google's warehouse scale computers: The numa experience," in *Proceedings of the 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, ser. HPCA '13. USA: IEEE Computer Society, 2013, p. 188–197. [Online]. Available: https://doi.org/10.1109/HPCA.2013.6522318

[20] P. Liu, "Convergence of high performance computing, big data, and machine learning applications on containerized infrastructures," Ph.D. dissertation, UPC, Departament d'Arquitectura de Computadors, Jul 2023. [Online]. Available: http://hdl.handle.net/2117/403550

[21] C. Wang, Y. Qiao, H. Ma, S. Liu, W. Chen, R. Netravali, M. Kim, and G. H. Xu, "Canvas: Isolated and adaptive swapping for Multi-Applications on remote memory," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 161–179. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/wang-chenxi

[22] E. Amaro, C. Branner-Augmon, Z. Luo, A. Ousterhout, M. K. Aguilera, A. Panda, S. Ratnasamy, and S. Shenker, "Can far memory improve job throughput?" in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3342195.3387522

[23] Keras, "Keras Applications," 2024. [Online]. Available: https://keras.io/api/applications/

[24] Apache, "ab - Apache HTTP server benchmarking tool," 2024. [Online]. Available: https://httpd.apache.org/docs/2.4/programs/ab.html

Authorized licensed use limited to: the Leddy Library at the University of Windsor. Downloaded on April 15,2026 at 22:51:55 UTC from IEEE Xplore. Restrictions apply.